



PROGRAMA:

Ingeniería de Sistemas

PERIODO ACADÉMICO:

2026-1

CURSO:

Arquitectura de software

CÓDIGO DEL CURSO:

07011438

CRÉDITOS:

4

SEMESTRE:

8°

ESTUDIANTES:

2022214053 - Rafael Andrés Acuña Rosales
2022214045 - Camilo Andrés Jiménez Carrillo
2022214005 - María Camila Gómez Amado
2022214066 - Karen Tatyana Mora Arévalo

Taller - Evaluación Arquitectura basada en ATAM

Evaluar la arquitectura de su proyecto presentando:

1. Sistema para evaluar: Describe el sistema que se va a revisar.

El sistema para evaluar es **AgroTech**, una plataforma digital de agricultura inteligente para pequeños productores del Magdalena. Esta plataforma permite a los agricultores gestionar sus fincas mediante módulos como: gestión de cultivos, finanzas, clima, tareas, reportes, recomendaciones.

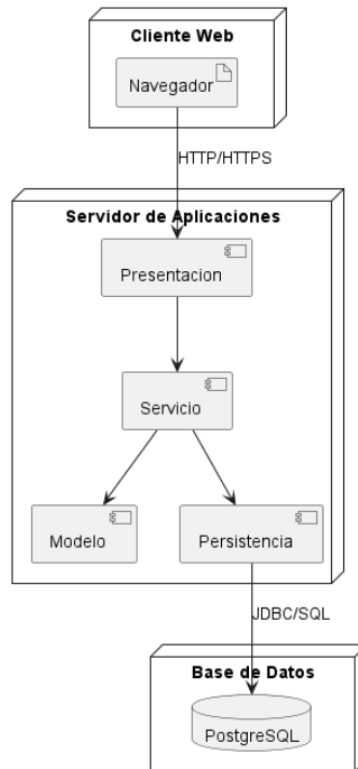
El sistema está diseñado para funcionar en contextos rurales con conectividad limitada, ayudando a mejorar la toma de decisiones agrícolas.

2. Descripción de la arquitectura: detalle el estilo arquitectónico aplicado con tecnología a usar (mostrar modelo de despliegue).

El sistema adopta un **estilo arquitectónico en capas**. Este estilo organiza el software en niveles jerárquicos donde cada capa tiene una responsabilidad específica y solo depende de la capa inmediatamente inferior. Las capas definidas son: presentación, lógica de negocio, acceso a datos y persistencia. Es soportado por tecnologías modernas orientadas al desarrollo web escalable y mantenible.

Tecnologías:

- **Frontend:** Next.js implementado como PWA (Progressive Web Application), permitiendo funcionamiento offline-first mediante almacenamiento local y sincronización posterior de datos. Esto mejora la disponibilidad y experiencia de usuario en zonas rurales con conectividad limitada.
- **Backend:** Spring Boot bajo arquitectura REST, permitiendo desacoplar la lógica de negocio del cliente.
- **Base de datos:** PostgreSQL, seleccionada por su estabilidad, soporte transaccional y capacidad de manejar grandes volúmenes de información estructurada.



3. Definir 5 atributos de calidad: seleccione de los 12 atributos y justifique su elección.

(<https://syndicode.com/blog/12-software-architecture-quality-attributes/>)

- **Modificabilidad:** El sistema debe permitir realizar cambios en funcionalidades o reglas de negocio con impacto mínimo en otras capas, gracias a la separación entre presentación, lógica de negocio y datos.
- **Disponibilidad:** El sistema debe garantizar el acceso a sus funcionalidades en la mayor parte del tiempo, incluso en condiciones de conectividad intermitente.
- **Usabilidad:** El sistema debe ser fácil de usar, con interfaces intuitivas y comprensibles para usuarios con bajo nivel de alfabetización digital.
- **Rendimiento:** El sistema debe responder en tiempos adecuados a las solicitudes de los usuarios, optimizando el uso de recursos en dispositivos de gama baja.
- **Seguridad:** El sistema debe proteger la información de los usuarios mediante mecanismos de autenticación, autorización y manejo seguro de datos.
- **Escalabilidad:** El sistema debe permitir un crecimiento moderado de usuarios y datos sin afectar significativamente el rendimiento, aunque la arquitectura en capas presenta limitaciones debido a la centralización de componentes y servicios

4. Define 2 escenarios posibles por cada atributo.

- **Modificabilidad:**
 - E1: Desarrollador – Se solicita agregar un nuevo tipo de recomendación – Durante mantenimiento – Se implementa en la capa de negocio sin afectar presentación ni datos.
 - E2: Desarrollador – Se requiere cambiar reglas de cálculo de recomendaciones – Durante operación – Se modifica solo la capa lógica.
- **Disponibilidad:**

E1: Usuario – Consulta información – Con conectividad intermitente – Sistema permite acceso a datos almacenados localmente.

E2: Sistema – Fallo temporal de red – Permite sincronización posterior.

- **Usabilidad:**

E1: Usuario novato – Accede por primera vez – Navega funciones básicas sin ayuda.

E2: Usuario – Registra cultivo – Completa proceso sin errores.

- **Rendimiento:**

E1: Usuario – Consulta cultivos – Sistema responde rápidamente.

E2: Usuario – Registra información – Baja conectividad – Procesamiento eficiente.

- **Seguridad:**

E1: Usuario no autorizado – Intenta acceder – Sistema bloquea acceso o no lo permite.

E2: Sistema – Manejo de datos sensibles - Información protegida.

- **Escalabilidad:**

E1: Múltiples productores agrícolas – Acceden simultáneamente al sistema – Durante temporada de alta actividad agrícola – El sistema mantiene tiempos de respuesta adecuados sin interrupciones.

E2: Sistema – Incremento en volumen de datos de cultivos y reportes – Durante crecimiento de usuarios – La plataforma soporta el aumento de información sin degradación crítica del rendimiento.

5. **Evaluar arquitectura. Siga este modelo**

- **Modificabilidad**

Escenario	Se cumple – Si/No/Parcial	Justificación técnica	Riesgo
Modificabilidad – E1: Agregar nuevo tipo de recomendación	Si	La arquitectura en capas permite agregar la nueva funcionalidad únicamente en la capa lógica de negocio, sin modificar la capa de presentación ni la capa de persistencia. La separación de responsabilidades facilita el cambio aislado.	Riesgo bajo. Si no se respetan los límites de capa, puede generarse acoplamiento entre capas, dificultando futuros cambios.

Escenario	Se cumple – Si/No/Parcial	Justificación técnica	Riesgo
Modificabilidad – E2: Cambiar reglas de cálculo de recomendaciones	Si	Las reglas de negocio están centralizadas en la capa de servicios. Modificar la lógica de cálculo solo requiere actualizar los componentes de servicio correspondiente, manteniendo intactas las otras capas.	Riesgo moderado. Un cambio en las reglas puede afectar sin querer otras funcionalidades que ya funcionaban correctamente, y sin pruebas previas, el error solo se detecta cuando el usuario lo reporta.

- **Disponibilidad**

Escenario	Se cumple – Si/No/Parcial	Justificación técnica	Riesgo
Disponibilidad – E1: Consulta con conectividad intermitente	Parcial	Next.js configurado como PWA con soporte offline-first permite almacenar datos en caché local. El usuario puede consultar información previamente sincronizada sin conexión activa.	Riesgo moderado. La información mostrada puede no estar actualizada si la última sincronización fue hace mucho tiempo. Se requiere gestión de conflictos en la sincronización.

Escenario	Se cumple – Si/No/Parcial	Justificación técnica	Riesgo
Disponibilidad – Fallo temporal de red y sincronización posterior	Parcial	La PWA encola las operaciones realizadas sin conexión y las sincronización automáticamente cuando se restablece la red. El <u>backend</u> debe soportar operaciones idempotentes para manejar reenvíos correctamente.	Riesgo alto. Sin un mecanismo robusto de resolución de conflictos, los datos sincronizados pueden sobrescribir cambios más recientes del servidor, causando inconsistencias.

- Usabilidad

Escenario	Se cumple – Si/No/Parcial	Justificación técnica	Riesgo
Usabilidad – E1: Usuario novato navega funciones básicas sin ayuda	Parcial	Next.js permite implementar interfaces responsivas con componentes guiados. Sin embargo, se requiere diseño UX específico para usuarios con baja alfabetización digital, incluyendo lenguaje sencillo e íconos representativos.	Riesgo moderado. Sin pruebas de usabilidad con usuarios reales del contexto rural, la interfaz puede no satisfacer las necesidades reales del perfil objetivo.

Escenario	Se cumple – Si/No/Parcial	Justificación técnica	Riesgo
Usabilidad – E2: Usuario registra cultivo sin errores	Si	El flujo de registro de cultivos puede implementarse en pasos secuenciales con validaciones en tiempo real en el <u>frontend</u> . La arquitectura en capas permite separar la validación de presentación de las reglas de negocio en <u>backend</u> .	Riesgo bajo. Las validaciones duplicadas (<u>frontend y backend</u>) aumentan la confiabilidad del proceso, aunque se debe garantizar consistencia entre ambas capas de validación.

- Rendimiento

Escenario	Se cumple – Si/No/Parcial	Justificación técnica	Riesgo
Rendimiento – E1: Usuario consulta cultivos y el sistema responde rápidamente	Parcial	La arquitectura por capas introduce latencia por el paso entre capas. La base de datos centralizada puede convertirse en cuello de botella ante múltiples usuarios concurrentes.	Riesgo alto. Sin optimización de consultas, el sistema puede volverse lento cuando muchos usuarios lo usen al mismo tiempo.

Escenario	Se cumple – Si/No/Parcial	Justificación técnica	Riesgo
Rendimiento – E2: Registro de información con baja conectividad	Parcial	El modo <u>offline</u> de la PWA permite registrar datos localmente con procesamiento mínimo en el dispositivo. Sin embargo, dispositivos de gama baja pueden experimentar lentitud al procesar datos locales.	Riesgo moderado. El procesamiento local en dispositivos de baja capacidad puede causar demoras.

- Seguridad

Escenario	Se cumple – Si/No/Parcial	Justificación técnica	Riesgo
Seguridad – E1: Usuario no autorizado intenta acceder al sistema	Si	Spring <u>Boot</u> soporta implementación de Spring Security con JWT para autenticación y autorización. Los endpoints pueden protegerse con roles y permisos. Next.js puede gestionar rutas protegidas en el <u>frontend</u> redirigiendo a usuarios no autenticados.	Riesgo moderado. Si los tokens JWT no se gestionan correctamente, un token comprometido puede permitir acceso no autorizado hasta su vencimiento.

Escenario	Se cumple – Si/No/Parcial	Justificación técnica	Riesgo
Seguridad – E2: Manejo seguro de datos sensibles de los usuarios	Parcial	Spring <u>Boot</u> implementa cifrado de contraseña con <u>BCrypt</u> y control de acceso por usuarios mediante Spring Security, garantizando que cada agricultor solo pueda visualizar su propia información. Por restricciones económicas del proyecto, la comunicación se realiza por HTTP.	Riesgo moderado. El uso de HTTP en lugar de HTTPS expone los datos en tránsito a posibles interceptaciones.

6. Identificación de los Trade-offs.

- **Rendimiento vs Modificabilidad:** Mejorar la separación por capas (más abstracción) facilita cambios, pero aumenta el tiempo de respuesta por el paso de las capas. Se gana mayor mantenimiento y evolución, pero se sacrifica el incremento en latencia de respuesta.
- **Disponibilidad vs Consistencia:** Permitir funcionamiento off-line mejora la disponibilidad en zonas rurales, pero puede generar inconsistencias temporales en los datos. Se gana un sistema usable, sin conexión, pero se sacrifica que los datos no siempre estén sincronizados en tiempo real.
- **Rendimiento vs Seguridad:** El uso de cifrado y validaciones incrementa la seguridad, pero puede afectar el tiempo de respuesta en dispositivos de gama baja. Se gana mayor protección de datos personales. Pero se pierde la rapidez en el procesamiento.
- **Usabilidad vs funcionalidad:** Una interfaz simple mejor la experiencia de un usuario rural, pero puede limitar la cantidad de funcionalidades visibles. Se gana la facilidad de uso, pero se sacrifica un menor acceso a funciones avanzadas.
- **Interoperabilidad vs Confidencialidad:** Depender de servicios externos (datos climáticos) permite mayor funcionalidad, pero introduce fallos si estos servicios no están disponibles. Se gana acceso de información externa valiosa, pero se sacrifica depender de terceros.
- **Adaptabilidad vs complejidad:** Hacer el sistema flexible a múltiples condiciones rurales mejora su uso en distintos contextos, pero aumenta la complejidad del desarrollo. Se gana una mayor cobertura en escenarios rurales, pero se requiere mayor conocimiento técnico para implementar aquella funcionalidad.

7.

8. Conclusiones

La arquitectura en capas propuesta es adecuada para la plataforma Digital Agrícola Inteligente, ya que facilita la organización del sistema, la mantenibilidad y su adaptación a entornos con conectividad limitada y dispositivos de bajo costo. Sin embargo, presenta limitaciones en términos de escalabilidad debido a la posible centralización de componentes y restricción de infraestructura. Además, la dependencia de

servicios externos puede afectar la confidencialidad del sistema. En general, la solución representa un equilibrio en costo, viabilidad técnica y atributos de calidad, siendo coherente con las necesidades y restricciones del contexto del proyecto.